

VelocityDash



Alan Bissio

E.E.S.T. N°1 Vicente López

7mo 2da

Proyecto de Implementación de Sitios Web Dinámicos

Jorge Guzman

22 de abril de 2026

Índice

1. Introducción.....	2
2. Objetivo de la actividad.....	3
3. Tecnologías utilizadas.....	4
3.1. Svelte.....	4
3.2. SvelteKit.....	4
3.3. TypeScript.....	4
3.4. Chart.js.....	5
3.5. WebSocket.....	5
3.6. html2canvas.....	5
3.7. jsPDF.....	6
4. Arquitectura de la solución.....	7
5. Dashboard de métricas.....	9
6. Visualización mediante gráficos.....	10
6.1. Gráfico de líneas.....	11
6.2. Gauge.....	11
6.3. Heatmap.....	11
6.4. Gráfico de barras.....	11
7. Actualización de datos en tiempo real.....	13
8. Animaciones e interacción de la interfaz.....	14
9. Store reactivo global.....	15
10. Filtros de rango temporal.....	16
11. Sistema de alertas y configuración de umbrales.....	16
12. Exportación del dashboard a PDF.....	18
13. Prueba de funcionamiento.....	19
14. Dificultades encontradas.....	20
15. Resultado final.....	21
16. Conclusión.....	22

1. Introducción

En esta actividad se desarrolló VelocityDash, un panel web destinado al análisis y monitoreo del rendimiento de aplicaciones web.

El proyecto fue desarrollado utilizando Svelte y SvelteKit, incorporando diferentes herramientas para representar información mediante gráficos, actualizar los datos en tiempo real y permitir la interacción con las métricas obtenidas.

El dashboard trabaja principalmente con cuatro métricas de rendimiento web: LCP, FID, CLS y TTFB. Estos valores se muestran mediante tarjetas informativas y diferentes tipos de gráficos, permitiendo observar tanto los valores actuales como el historial de mediciones.

Uno de los principales objetivos del proyecto fue implementar una interfaz dinámica, en la cual los datos pudieran cambiar automáticamente sin necesidad de recargar la página. Para esto se utilizó una conexión WebSocket entre el navegador y un servidor encargado de generar y enviar nuevas mediciones.

Además, se incorporaron animaciones utilizando las herramientas de Svelte, un sistema de alertas configurable, filtros de rango temporal y la posibilidad de exportar el dashboard completo a un archivo PDF.

2. Objetivo de la actividad

El objetivo principal de la actividad fue desarrollar un dashboard web capaz de mostrar información de rendimiento de manera dinámica e interactiva.

La implementación debía permitir:

- Mostrar las métricas actuales de rendimiento.
- Representar los datos mediante diferentes tipos de gráficos.
- Actualizar las métricas automáticamente mediante WebSocket.
- Incorporar animaciones en la interfaz.
- Centralizar la información utilizando un store reactivo.
- Filtrar los datos según diferentes rangos temporales.
- Configurar umbrales para detectar valores fuera de rango.
- Mostrar alertas cuando una métrica supere el límite establecido.
- Exportar el contenido del dashboard a un archivo PDF.

De esta manera, se buscó desarrollar una aplicación que no solamente mostrara información estática, sino que pudiera actualizar, procesar y representar los datos de manera dinámica.

3. Tecnologías utilizadas

Para realizar el proyecto se utilizaron diferentes tecnologías y librerías.

3.1. Svelte

Svelte fue utilizado para desarrollar los componentes de la interfaz.

La aplicación está dividida en diferentes componentes independientes, lo que permite separar las distintas partes del dashboard y facilitar su mantenimiento.

Entre los componentes desarrollados se encuentran las tarjetas de métricas, gráficos, filtros, alertas y botón de exportación.

3.2. SvelteKit

SvelteKit fue utilizado como framework principal del proyecto.

Permite organizar la aplicación mediante rutas y componentes, además de proporcionar la estructura necesaria para ejecutar el proyecto durante el desarrollo.

La página principal del dashboard se encuentra implementada en:

`src/routes/+page.svelte`

Los componentes reutilizables se encuentran organizados dentro de:

`src/lib/components/`

Mientras que los stores, servicios y tipos utilizados por la aplicación se encuentran separados en sus correspondientes carpetas.

3.3. TypeScript

TypeScript fue utilizado para definir los tipos de datos empleados por la aplicación.

Se creó una estructura específica para representar las métricas:

- LCP
- FID
- CLS
- TTFB
- Timestamp

También se definieron los posibles estados de cada métrica:

- good
- warning
- bad

Esto permite mantener una estructura clara para los datos y reducir errores durante el desarrollo.

3.4. Chart.js

Chart.js fue utilizado para crear los gráficos interactivos del dashboard.

La librería permite representar las métricas mediante diferentes tipos de gráficos y actualizar su contenido cuando cambian los datos.

En el proyecto se utilizaron gráficos de líneas, barras y un gráfico tipo gauge basado en un gráfico de doughnut.

3.5. WebSocket

WebSocket fue utilizado para implementar la actualización de información en tiempo real.

A diferencia de una comunicación tradicional en la que el cliente debe realizar solicitudes para obtener información nueva, WebSocket mantiene una conexión abierta entre el cliente y el servidor.

Esto permite que el servidor envíe nuevas métricas automáticamente.

En el proyecto se creó un servidor independiente ubicado en:

server/server.mjs

El servidor genera nuevas mediciones y las envía aproximadamente cada tres segundos.

3.6. html2canvas

html2canvas fue utilizado para capturar visualmente el contenido del dashboard.

Al momento de realizar una exportación, la librería toma el contenido de la sección correspondiente de la página y lo convierte en una imagen que posteriormente puede incorporarse al PDF.

3.7. jsPDF

jsPDF fue utilizado para generar el archivo PDF final.

La imagen generada mediante html2canvas se incorpora al documento y luego el archivo se descarga automáticamente con el nombre:

velocitydash-report.pdf

4. Arquitectura de la solución

La aplicación fue organizada separando la interfaz, la lógica de datos y los servicios utilizados para la comunicación.

La estructura principal del proyecto es:

src/

```

├── lib/
|   ├── components/
|   |   ├── charts/
|   |   ├── dashboard/
|   |   └── ui/
|   ├── services/
|   ├── stores/
|   └── types/
|
└── routes/
    ├── +layout.svelte
    └── +page.svelte
  
```

server/

```

└── server.mjs
  
```

os componentes ubicados en **dashboard** se encargan principalmente de las métricas principales, mientras que la carpeta **charts** contiene los diferentes gráficos.

Los componentes ubicados en **ui** contienen elementos de interacción como el filtro temporal, las alertas, la configuración de umbrales y la exportación a PDF.

Los datos se centralizan mediante los stores ubicados en:

`src/lib/stores/`

Por otra parte, la comunicación WebSocket se encuentra separada en un servicio:

`src/lib/services/websocket.ts`

De esta forma, la interfaz no necesita encargarse directamente de toda la lógica de comunicación.

5. Dashboard de métricas

El elemento principal de VelocityDash es el dashboard de rendimiento.

En la parte superior se muestran cuatro tarjetas correspondientes a las métricas principales:

- **LCP**
- **FID**
- **CLS**
- **TTFB**

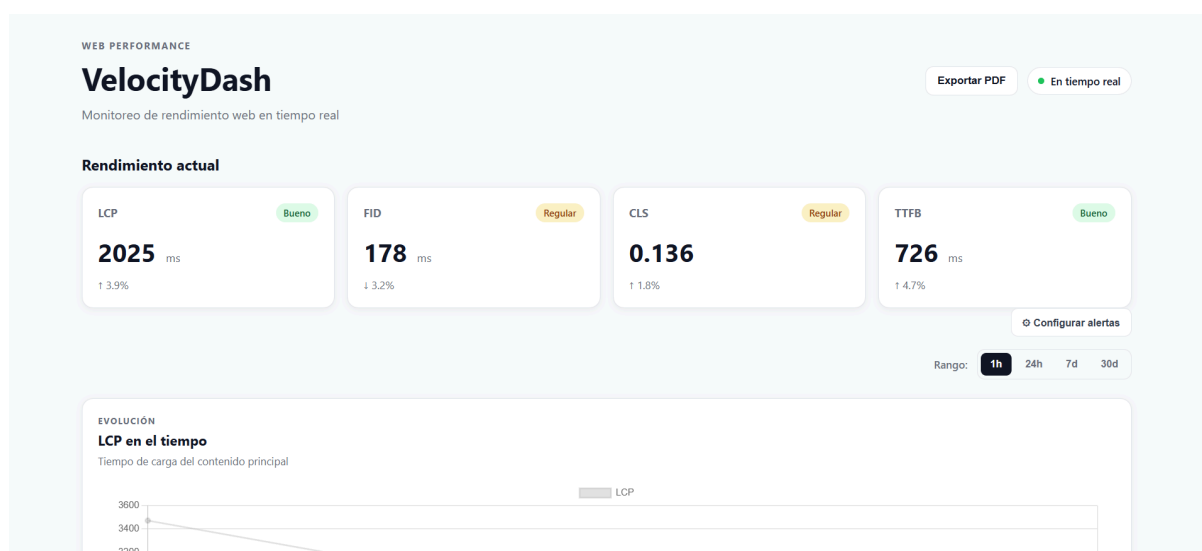
Cada tarjeta muestra el valor actual de la métrica y su estado correspondiente.

Los estados utilizados son:

- **Good:** el valor se encuentra dentro del rango considerado correcto.
- **Warning:** el valor se encuentra en un rango intermedio.
- **Bad:** el valor supera el límite establecido.

Los estados son calculados automáticamente a partir de los valores recibidos.

También se muestran indicadores de tendencia para proporcionar información adicional sobre el comportamiento de las métricas.



Las métricas no permanecen estáticas. Cada vez que se recibe una nueva medición mediante WebSocket, los valores mostrados en el dashboard se actualizan.

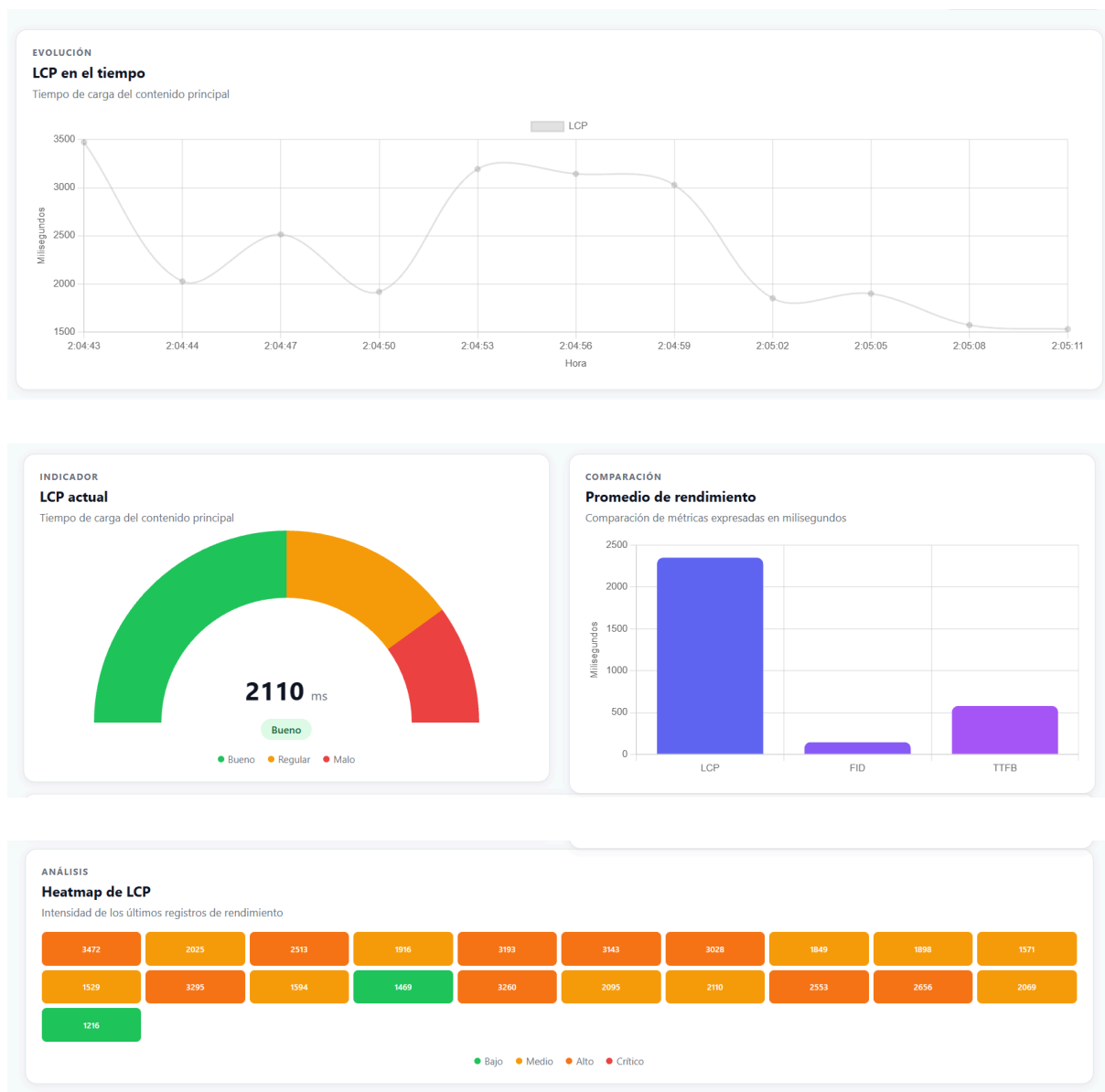
6. Visualización mediante gráficos

Además de las tarjetas principales, el proyecto incorpora diferentes representaciones gráficas para facilitar la interpretación de los datos.

Se implementaron cuatro tipos de visualizaciones:

1. Gráfico de líneas.
2. Gauge.
3. Heatmap.
4. Gráfico de barras.

Cada uno representa la información de una manera diferente.



6.1. Gráfico de líneas

El gráfico de líneas permite observar la evolución del **LCP** a través de las diferentes mediciones recibidas.

Cada nueva medición se incorpora al historial almacenado en el store.

Esto permite visualizar cómo fue variando el valor de LCP con el paso del tiempo.

El gráfico se actualiza automáticamente cuando se incorporan nuevos datos.

6.2. Gauge

El gráfico tipo **gauge** permite representar visualmente el valor actual de LCP.

Para implementarlo se utilizó un gráfico de tipo doughnut de Chart.js configurado para representar únicamente una parte del círculo.

De esta manera se obtiene una representación similar a un indicador de velocidad, permitiendo interpretar rápidamente el estado de la métrica.

6.3. Heatmap

El **heatmap** representa el historial de métricas utilizando diferentes niveles visuales.

Cada celda representa una medición y su apariencia cambia dependiendo del valor obtenido.

Esta representación permite observar rápidamente períodos donde los valores fueron más altos o más bajos.

A diferencia de los gráficos tradicionales, el heatmap permite visualizar una mayor cantidad de mediciones de manera compacta.

6.4. Gráfico de barras

El gráfico de barras permite comparar los valores promedio de diferentes métricas.

Se utilizan los valores de:

- LCP
- FID
- TTFB

Los promedios son calculados mediante stores derivados a partir del historial de mediciones.

Esto permite comparar las métricas en una misma representación visual.

7. Actualización de datos en tiempo real

Una de las funcionalidades principales del proyecto es la actualización automática de las métricas.

Para implementar esta funcionalidad se desarrolló un servidor WebSocket independiente.

El servidor genera valores simulados para:

- LCP
- FID
- CLS
- TTFB

Cada medición también contiene un timestamp que permite identificar el momento en el que fue generada.

Una vez establecida la conexión, el servidor envía una medición inicial y posteriormente genera nuevas mediciones aproximadamente cada tres segundos.

El cliente establece la conexión mediante el servicio:

`src/lib/services/websocket.ts`

Cuando llega un nuevo mensaje, el contenido recibido se convierte desde JSON a la estructura correspondiente y se actualizan los datos del dashboard.

Una prueba importante realizada durante el desarrollo consistió en desconectar el servidor WebSocket. Al hacerlo, las métricas dejaron de actualizarse, demostrando que la actualización del dashboard depende de los datos recibidos mediante la conexión.

8. Animaciones e interacción de la interfaz

Se incorporaron diferentes animaciones utilizando las herramientas de transición de Svelte.

Las animaciones se utilizan principalmente para mejorar la aparición de elementos y los cambios visuales de la interfaz.

Se utilizaron transiciones como:

- `fade`
- `fly`

Las tarjetas de métricas utilizan transiciones de aparición, mientras que los gráficos también cuentan con animaciones al ingresar en la página.

Además, los cambios de estado de las métricas utilizan una transición `fade`, permitiendo que los cambios entre los estados visuales se realicen de manera progresiva.

Estas animaciones permiten que la interfaz sea más dinámica sin modificar la funcionalidad principal del dashboard.

9. Store reactivo global

Para centralizar el manejo de información se implementó un sistema de stores utilizando los stores reactivos de Svelte.

El store principal contiene el estado actual de las métricas:

`metrics`

También se creó un historial:

`metricHistory`

Este historial almacena las mediciones recibidas y permite utilizar los datos para los gráficos y cálculos.

Además, se implementaron stores derivados para obtener diferentes valores calculados, entre ellos:

- Promedio de LCP.
- Promedio de FID.
- Promedio de CLS.
- Promedio de TTFB.
- Estado general del dashboard.
- Historial filtrado según el rango seleccionado.

Cuando llega una nueva métrica mediante WebSocket, la función:

`updateMetrics()`

actualiza el estado actual y agrega la nueva medición al historial.

De esta manera, diferentes componentes pueden utilizar los mismos datos sin tener que mantener copias independientes de la información.

10. Filtros de rango temporal

El dashboard incorpora un selector de rango temporal que permite elegir entre:

- 1h
- 24h
- 7d
- 30d

El rango seleccionado se almacena en un store y se utiliza para determinar qué parte del historial debe mostrarse.

La aplicación calcula un momento inicial dependiendo del rango elegido y filtra las mediciones cuyo timestamp se encuentre dentro de ese período.

Esto permite que el usuario pueda cambiar el período de análisis sin modificar directamente los datos originales almacenados.

El sistema está preparado para trabajar con períodos más extensos a medida que exista una cantidad suficiente de datos históricos.



11. Sistema de alertas y configuración de umbrales

Se implementó un sistema de alertas para detectar automáticamente cuando una métrica supera un límite determinado.

Los umbrales iniciales utilizados son:

Métrica	Umbral
LCP	4000 ms
FID	300 ms
CLS	0.25
TTFB	1800 ms

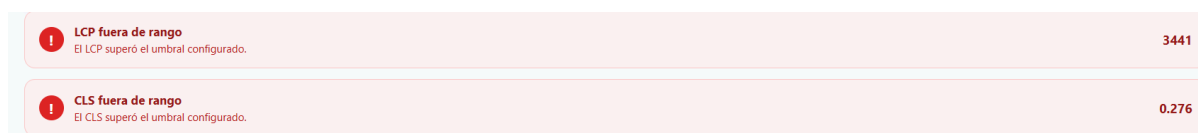
Estos valores pueden modificarse desde la interfaz mediante el componente de configuración de alertas.

Cuando una métrica supera su umbral, el sistema genera una alerta indicando:

- La métrica afectada.
- El valor actual.
- El motivo de la alerta.

Por ejemplo, si el valor de LCP supera el umbral configurado, se muestra un mensaje indicando que el LCP se encuentra fuera de rango.

Las alertas utilizan una animación **fly** para aparecer visualmente en el dashboard.



La configuración permite modificar los valores de LCP, FID, CLS y TTFB y guardar los nuevos límites.

Una vez guardados, las alertas comienzan a utilizar los nuevos valores automáticamente.

12. Exportación del dashboard a PDF

Como última funcionalidad se incorporó la posibilidad de exportar el dashboard completo a un documento PDF.

Para esto se utilizaron las librerías **html2canvas** y **jsPDF**.

El proceso de exportación funciona de la siguiente manera:

1. El usuario presiona el botón **Exportar PDF**.
2. Se identifica la sección del dashboard que debe exportarse.
3. html2canvas genera una captura del contenido.
4. La captura se convierte en una imagen.
5. jsPDF crea un documento en formato A4.
6. La imagen se incorpora al documento.
7. El archivo se descarga automáticamente.

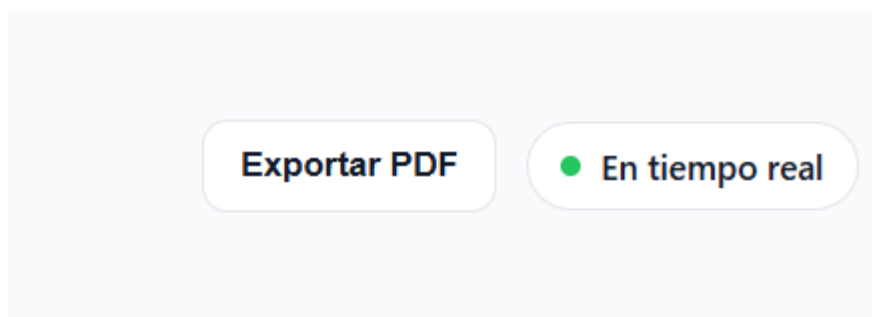
El archivo generado recibe el nombre:

velocitydash-report.pdf

También se agregó un estado visual en el botón mientras el archivo está siendo generado:

Generando PDF...

Esto evita que el usuario interprete que la aplicación no respondió mientras se procesa la exportación.



13. Prueba de funcionamiento

Para comprobar el funcionamiento de la aplicación se realizaron diferentes pruebas sobre las funcionalidades desarrolladas.

En primer lugar se verificó que el dashboard cargara correctamente y mostrara las tarjetas correspondientes a las cuatro métricas.

Luego se comprobó el funcionamiento de los cuatro gráficos y su actualización a medida que se recibían nuevas mediciones.

También se comprobó la conexión WebSocket. Se verificó que mientras el servidor permaneciera conectado las métricas continuaran actualizándose y que al desconectar la conexión dejaran de recibirse nuevos valores.

Se probaron las animaciones de aparición de las tarjetas y gráficos y los cambios visuales de los estados de las métricas.

También se comprobó el selector de rangos temporales y el funcionamiento del sistema de alertas modificando los umbrales desde la configuración.

Finalmente se verificó la generación del documento PDF y la descarga del archivo con el contenido del dashboard.

14. Dificultades encontradas

Durante el desarrollo se presentaron diferentes problemas que fue necesario solucionar para poder completar las funcionalidades.

Uno de los problemas apareció durante la implementación de las animaciones de Svelte. En una primera versión se utilizó una transición sin importar correctamente `fade`, lo que provocaba un error en la aplicación.

El problema se solucionó importando correctamente las transiciones utilizadas desde:

```
svelte/transition
```

Otro problema apareció durante la implementación de las alertas. Debido al uso de **Svelte 5**, algunas variables locales del componente necesitaban utilizar el sistema de estado correspondiente.

El uso de variables normales provocaba advertencias indicando que sus modificaciones no generarían correctamente una actualización de la interfaz.

Esto se solucionó utilizando:

```
$state(...)
```

para las variables locales utilizadas por la configuración de alertas.

También fue necesario corregir la distribución de los gráficos. Inicialmente algunos gráficos no se adaptaban correctamente al espacio disponible y uno de ellos podía quedar desplazado fuera de la pantalla.

Se solucionó utilizando un sistema de `grid` con columnas flexibles y estableciendo límites de ancho para los componentes.

Otro aspecto que requirió pruebas fue la conexión WebSocket, especialmente para comprobar que los datos realmente dependieran de la conexión y no de un temporizador independiente dentro del frontend.

Después de las modificaciones se verificó que, al desconectar el WebSocket, las métricas dejaran de recibir actualizaciones.

15. Resultado final

Como resultado final se obtuvo un dashboard web funcional para el monitoreo de métricas de rendimiento.

Actualmente VelocityDash permite:

- Mostrar LCP, FID, CLS y TTFB.
- Indicar el estado de cada métrica.
- Mantener un historial de mediciones.
- Mostrar un gráfico de líneas.
- Mostrar un gauge.
- Mostrar un heatmap.
- Mostrar un gráfico de barras.
- Recibir datos mediante WebSocket.
- Actualizar las métricas automáticamente.
- Utilizar animaciones de Svelte.
- Centralizar los datos mediante stores reactivos.
- Calcular valores promedio.
- Seleccionar diferentes rangos temporales.
- Configurar umbrales para las métricas.
- Mostrar alertas cuando los valores superan dichos umbrales.
- Exportar el dashboard a un documento PDF.

El resultado integra todas las funcionalidades desarrolladas durante las diferentes etapas del proyecto en una única interfaz.

16. Conclusión

El desarrollo de VelocityDash permitió aplicar diferentes conceptos relacionados con la creación de sitios web dinámicos utilizando Svelte y SvelteKit.

A lo largo del proyecto se trabajó con componentes reutilizables, stores reactivos, gráficos, comunicación WebSocket, transiciones y generación de documentos.

La utilización de WebSocket permitió implementar una actualización de datos en tiempo real, mientras que los stores de Svelte permitieron centralizar la información y compartirla entre los diferentes componentes del dashboard.

Los gráficos permitieron representar las métricas de diferentes maneras y facilitar su interpretación. Por otro lado, el sistema de alertas permitió agregar una respuesta visual ante valores que superaran determinados umbrales.

Finalmente, la incorporación de los filtros temporales y la exportación a PDF permitió ampliar las posibilidades de interacción con la información presentada.

En conjunto, el proyecto permitió integrar diferentes tecnologías y conceptos en una aplicación web dinámica, modular y orientada al análisis de datos de rendimiento.